DOCUMENT 02

Technical Requirements Document

TRD - architecture, stack, and engineering decisions

Study Topic Extractor

Version: 1.0
Status: Initial Specification
Date: September 28, 2026

Turn long study material into a focused study roadmap.

Table of Contents

1. Technical Overview 2

1.1 Architecture Principles 2

2. System Architecture 3

2.1 Technology Evaluation and Roles 3

3. Frontend Architecture 5

4. Backend Architecture 6

4.1 Layer Rules 6

5. NLP Architecture 7

6. File Processing Architecture 8

7. API Architecture 9

8. Authentication and Authorization Architecture 10

9. Storage Architecture 11

10. Error Handling, Logging, Monitoring 12

10.1 Error Handling 12

10.2 Logging 12

10.3 Monitoring 12

11. Caching, Performance, Scalability 13

11.1 Caching 13

11.2 Performance Targets 13

11.3 Scalability Path 13

12. Security Architecture Summary 14

13. Deployment Summary 15

14. Configuration Reference 16

14.1 Numeric Limits 16

1. Technical Overview

Study Topic Extractor is a three-tier web application: a Next.js frontend, a stateless FastAPI backend that hosts the NLP pipeline, and (from Phase 2) a Supabase-managed PostgreSQL database with Auth and Storage. The MVP is stateless: no database, no accounts, no server-side persistence of user content.

1.1 Architecture Principles

- **Explainable first:** classical NLP (spaCy, TF-IDF, n-grams) is the core; embeddings and LLMs augment it.
- **Deterministic by default:** identical input, configuration and `pipeline_version` produce identical output.
- **Stateless API:** any instance can serve any request; state lives in the DB or the client.
- **Modular services:** each pipeline stage is one module with typed inputs and outputs.
- **Privacy by design:** process in memory, never log document text, delete temporary data.
- **Optional heavy dependencies:** Sentence Transformers, OCR and LLMs are behind feature flags.

2. System Architecture

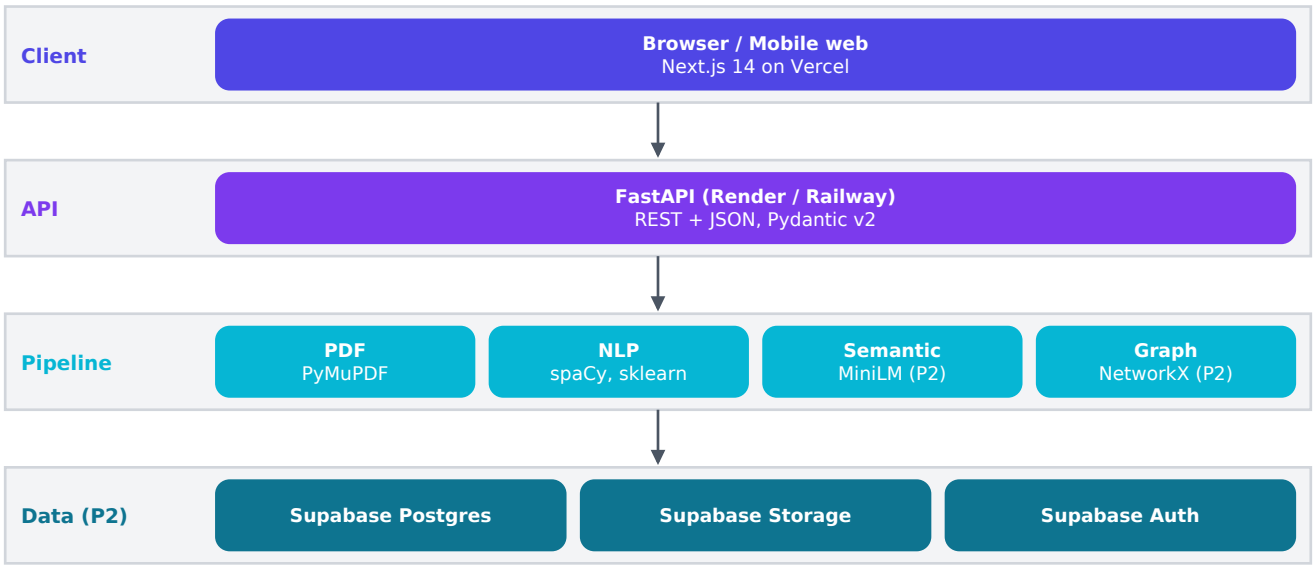


Figure 1. Logical architecture (P2 = Phase 2)

2.1 Technology Evaluation and Roles

Layer	Choice	Role	Alternatives considered / decision
Frontend framework	Next.js 14 (App Router) + TypeScript	Routing, SSR for landing page, typed UI	Plain React (no SSR/routing), Streamlit (fast MVP but poor design control). Next.js chosen; Streamlit acceptable only as a throwaway prototype.
Styling	Tailwind CSS + CSS variables tokens	Design system, dark/light themes	CSS-in-JS rejected (runtime cost)
Charts / graph	Plotly.js (score charts), Cytoscape.js (graph)	Distribution chart; interactive relationship graph	D3 raw (too low-level); one library for both rejected (Plotly weak for graphs)
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic v2	REST API, validation, OpenAPI docs	Flask (no built-in validation), Django (heavy)
Tokenization / lemmas / POS	spaCy `en_core_web_sm`	Sentences, tokens, lemmas, POS, noun chunks	NLTK alone (weaker POS/chunking)
Stopwords supplement	NLTK stopwords corpus	Extends spaCy stopword list with academic filler	Kept minimal: NLTK used only for its stopword list
Statistics / ML	scikit-learn	TfidfVectorizer, KMeans, AgglomerativeClustering	Custom TF-IDF rejected (reuse tested code)
Embeddings (P2)	Sentence Transformers `all-MiniLM-L6-v2`	384-d embeddings for similarity and clustering	Larger models rejected (memory/latency)
Graph logic (P2)	NetworkX	Hierarchy arborescence, graph JSON	igraph rejected (extra native dependency)
PDF	PyMuPDF (fitz)	Fast text and layout extraction, page count, encryption check	pdfminer (slower), pypdf (weaker layout)
OCR (P2)	Tesseract via pytesseract	Scanned pages	Cloud OCR rejected (cost, privacy)

Layer	Choice	Role	Alternatives considered / decision
Export	ReportLab (PDF), csv/json stdlib	Server-side file generation	Client-side PDF rejected (inconsistent output)
Database (P2)	PostgreSQL on Supabase	Relational storage, RLS	MongoDB rejected (relational data)
Auth (P2)	Supabase Auth (JWT)	Email/password and Google sign-in	Custom auth rejected (security risk)
Storage (P2)	Supabase Storage (private bucket)	Original PDFs	Local disk rejected (ephemeral hosts)
ORM / migrations (P2)	SQLAlchemy 2.0 + Alembic	Typed models, versioned schema	-
Rate limiting	SlowAPI (in-memory MVP; Redis later)	Per-IP / per-user limits	-
Testing	pytest, Vitest, Playwright, axe-core	Unit, integration, E2E, a11y	-
Deploy	Vercel, Render (Docker), Supabase	Hosting	Railway equivalent to Render

3. Frontend Architecture

- **Routing:** `/` landing, `/analyze`, `/results/[analysisId]`, `/focus/[analysisId]` (P2), `/history` (P2), `/dashboard` (P2), `/settings`, `/login` (P2).
- **State:** React state + Context for theme and settings; TanStack Query for API calls (retry, cache, loading states).
- **MVP result persistence:** result JSON kept in `sessionStorage` keyed by `analysisId`; cleared when the tab closes. Nothing is stored server-side.
- **API client:** single `lib/api.ts` generated types from OpenAPI (`openapi-typescript`) so frontend and backend contracts stay synchronized.
- **Components:** presentational components under `components/ui`; feature components under `components/analyze|results|graph|focus`.
- **Theming:** CSS variables defined in `styles/tokens.css`; `data-theme="light|dark"` on `<html>`; system preference default.
- **Accessibility:** semantic HTML, focus management on route change, `aria-live` regions for processing status.

4. Backend Architecture

FastAPI application with routers, dependency-injected services, and a pipeline orchestrator. CPU-bound NLP work runs in a thread pool (`run_in_threadpool`) so the event loop remains responsive. Models (spaCy, MiniLM) load once during the application lifespan and are shared read-only.

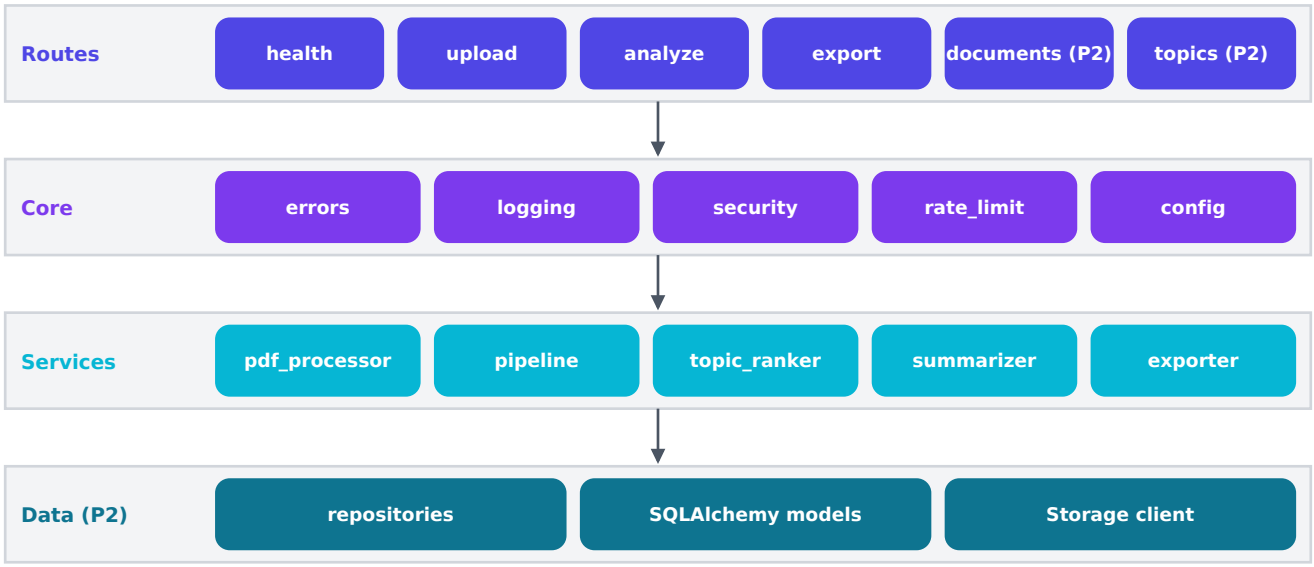


Figure 2. Backend module layers

4.1 Layer Rules

- Routes validate input (Pydantic), call one service function, map exceptions to the error format. No business logic in routes.
- Services are pure where possible: text in, structured data out; no HTTP or DB imports in NLP modules.
- Repositories are the only code that touches the database; every query takes `user_id`.
- Configuration is read once from environment (`pydantic-settings`); no hard-coded thresholds or weights (`config.py` defaults).

5. NLP Architecture

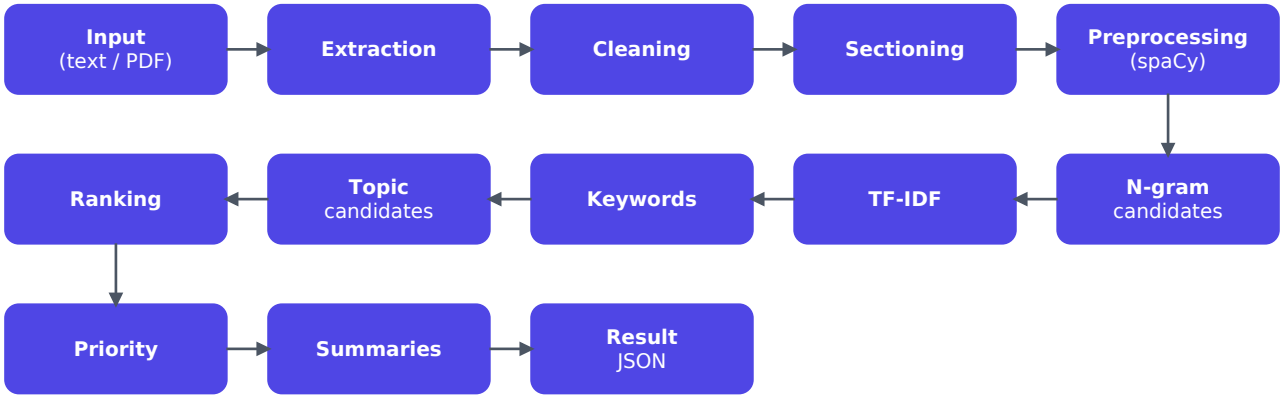


Figure 3. MVP pipeline (see NLP/AI Specification for detail)

Phase 2 inserts Semantic embeddings, Clustering, Hierarchy and Graph stages (Figure 4). Stages are registered in `pipeline.py` as an ordered list; Phase 2 stages are skipped when `ENABLE_SEMANTIC=false`, in which case the semantic weight is redistributed proportionally across the remaining signals.

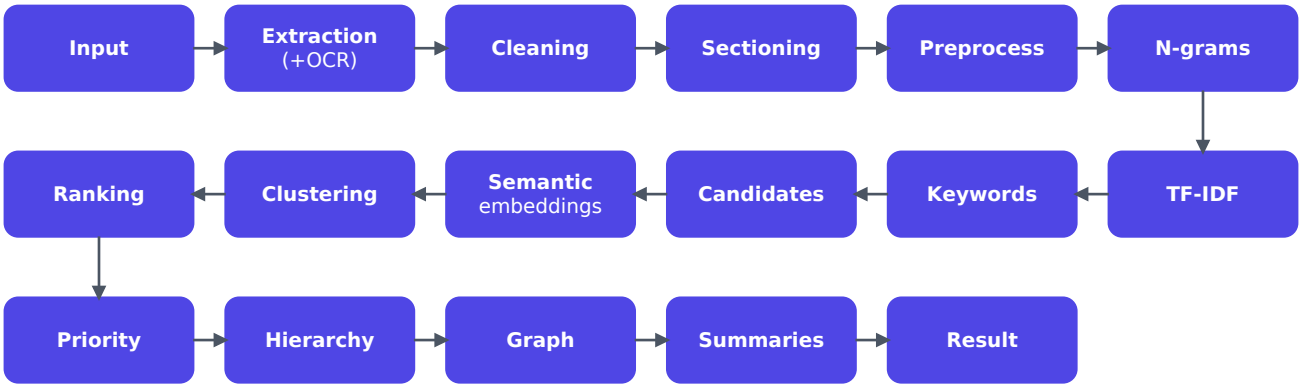


Figure 4. Full pipeline (Phase 2)

6. File Processing Architecture

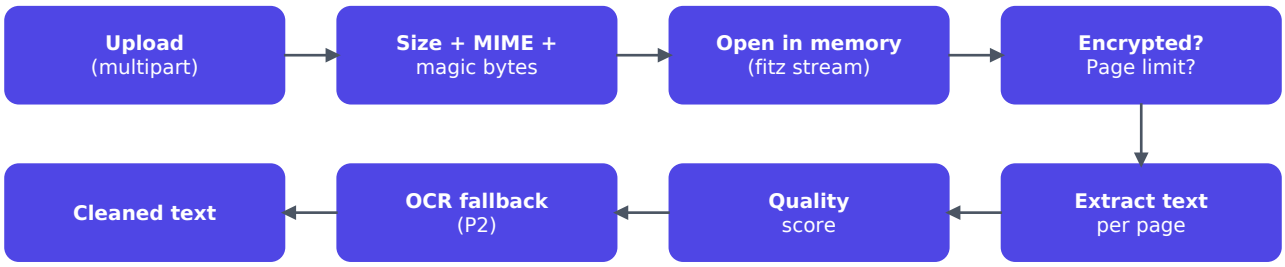


Figure 5. PDF ingestion

- PDFs are read as bytes and opened with `fitz.open(stream=..., filetype="pdf")`. In MVP no file is written to disk.
- **Extraction quality** = fraction of pages with at least 50 extractable characters, adjusted by ratio of alphabetic characters. Below 0.5, return warning `LOW_EXTRACTION_QUALITY`; if no text at all, error `NO_READABLE_TEXT` (or OCR in Phase 2).
- Headers, footers and page numbers repeated on more than 60% of pages are removed during cleaning.

7. API Architecture

REST over HTTPS, JSON bodies, `multipart/form-data` for uploads. Versioning via the `pipeline_version` field in results; URL versioning (`/api/v1`) is reserved but not used in MVP. OpenAPI schema is generated by FastAPI and is the contract for the frontend type generation. Full details are in the API Specification.

Method	Path	Phase	Auth	Purpose
GET	/api/health	MVP	None	Liveness and readiness check
POST	/api/upload	MVP	Optional	Validate a PDF and extract text (stateless in MVP; persists when authenticated)
POST	/api/analyze	MVP	Optional	Run the NLP pipeline on text or a stored document
POST	/api/export	MVP	None	Render an analysis result to PDF, CSV, TXT, MD or JSON (stateless)
GET	/api/documents	Phase 2	Required	List the caller's saved documents
GET	/api/documents/{id}	Phase 2	Required	Document metadata plus latest analysis result
DELETE	/api/documents/{id}	Phase 2	Required	Delete document, file, and all derived data
GET	/api/topics/{id}	Phase 2	Required	Topic detail with relations and questions
POST	/api/documents/{id}/questions	Phase 2	Required	Generate possible questions for topics
POST	/api/documents/{id}/export	Phase 2	Required	Export the stored analysis in a chosen format
GET	/api/analysis-runs/{id}	Phase 2	Required	Poll status of an asynchronous analysis
POST	/api/documents/{id}/sessions	Phase 2	Required	Record a study session
GET / PATCH	/api/settings	Phase 2	Required	Read or update user settings
DELETE	/api/me	Phase 2	Required	Delete the account and all data

8. Authentication and Authorization Architecture

- **MVP:** no authentication. Endpoints are stateless and rate limited by IP.
- **Phase 2:** Supabase Auth issues JWTs; frontend uses `@supabase/ssr`; backend verifies the JWT signature and expiry (`sub` = user id) in a `get_current_user` dependency.
- **Authorization:** ownership enforced in repositories (`WHERE user_id = :uid`) and by Postgres RLS as defense in depth.
- **Optional auth:** `/api/upload` and `/api/analyze` remain usable anonymously; supplying a valid token plus `save=true` persists results.

9. Storage Architecture

Data	MVP	Phase 2
Pasted text	Request body only; not stored	Stored in `documents.extracted_text` if `save=true`
PDF bytes	Memory only; released after extraction	Supabase Storage private bucket `documents`, path `{user_id}/{document_id}.pdf`
Analysis result	Returned to client; in-memory LRU cache (32 entries, 1 hour) keyed by content hash + config hash	`analysis_runs`, `topics`, `keywords`, relations tables
Exports	Generated per request, streamed	Same; optional cached copy not stored
User settings	Browser `localStorage`	`users.settings` JSONB

10. Error Handling, Logging, Monitoring

10.1 Error Handling

All errors use one envelope: `{"error": {"code", "message", "details", "request_id"}}`. Domain exceptions (`AppError` subclasses) carry a code and HTTP status; a global handler converts them. Unhandled exceptions become `PROCESSING_FAILED` (500) with a `request_id`; stack traces go to logs only.

Code	HTTP	User message	Trigger
INVALID_REQUEST	400	The request could not be understood.	Schema validation failed
INVALID_PDF	400	Please upload a valid PDF file.	Corrupted or non-PDF bytes
INVALID_FILE_TYPE	415	This file type is not supported. Please upload a PDF.	MIME or magic-byte mismatch
PASSWORD_PROTECTED_PDF	422	This PDF is password protected. Remove the password and try again.	Encrypted PDF
NO_READABLE_TEXT	422	We couldn't find readable text in this document.	Empty or scanned PDF without OCR
TEXT_TOO_SHORT	422	Please provide at least 200 characters of study material.	Below minimum length
UNAUTHORIZED	401	Please sign in to continue.	Missing or invalid token
FORBIDDEN	403	You don't have access to this resource.	Ownership check failed
NOT_FOUND	404	We couldn't find what you were looking for.	Unknown or foreign resource id
FILE_TOO_LARGE	413	This file is larger than the 10 MB upload limit.	PDF above 10 MB
DOCUMENT_TOO_LARGE	413	This document is larger than the supported processing limit.	Text or pages above limits
RATE_LIMITED	429	Too many requests. Please wait a moment and try again.	Rate limit exceeded
PROCESSING_FAILED	500	We couldn't analyze this document. Please try again.	Unhandled pipeline failure
SERVICE_UNAVAILABLE	503	The service is temporarily unavailable. Please try again shortly.	Dependency down or model loading

10.2 Logging

- Structured JSON logs (structlog): `timestamp, level, request_id, route, status, latency_ms, char_count, page_count, error_code`.
- **Never log** document text, extracted text, filenames of user documents, tokens, or API keys.
- Request ID generated per request, returned in `X-Request-ID`.

10.3 Monitoring

- Uptime check on `GET /api/health` (returns model-loaded status and version).
- Render/Railway metrics for CPU, memory, restarts; alert when memory > 85%.
- Optional Sentry with request bodies scrubbed (`send_default_pii=false`, `before_send` strips bodies).
- Pipeline stage timings recorded in `stats.processing_ms` and per-stage debug logs.

11. Caching, Performance, Scalability

11.1 Caching

- Model objects cached at startup (spaCy, MiniLM, background IDF table).
- Result cache: in-process LRU keyed by `sha256(normalized_text) + sha256(config)`; memory-only for privacy. Redis is optional in Phase 2 for multi-instance sharing.
- Frontend: TanStack Query cache for document lists; static assets on Vercel CDN.

11.2 Performance Targets

Scenario	Target (1 vCPU, MVP)
Text analysis, up to 5,000 characters	< 3 s
Text analysis, up to 30,000 characters	< 8 s
Text analysis, up to 150,000 characters	< 30 s (Phase 2: async job)
PDF extraction, up to 50 pages native	< 5 s
PDF extraction, up to 200 pages native	< 10 s
Phase 2 semantic stage overhead	+2 to +5 s depending on candidate count
Export generation	< 3 s
Frontend interaction (filters, tabs)	< 100 ms perceived

11.3 Scalability Path

1. Horizontal scaling: stateless API behind the host load balancer.
2. Background processing: FastAPI BackgroundTasks first; then Redis + RQ/Celery worker for large documents.
3. Model isolation: move embeddings to a dedicated worker or inference service if memory pressure grows.
4. Multilingual: language detection stage plus per-language spaCy model registry (`LANG_MODELS`).
5. Database: indexes and partitioning of `topics` by `analysis_run_id` if volume grows.

DEPLOYMENT CONSTRAINT. Sentence Transformers pull in PyTorch. Use the CPU-only wheel, load the model once, and provision at least 1 GB RAM (2 GB recommended) for Phase 2 backends. The MVP backend (no torch) fits in 512 MB.

12. Security Architecture Summary

Security is specified in detail in the Security Specification. Architectural highlights: TLS everywhere; CORS allow-list; file validation by extension, MIME and magic bytes; size and page limits; rate limiting; JWT verification; RLS; private storage; no content logging; secrets only in environment variables; prompt-injection isolation for optional LLM calls.

13. Deployment Summary

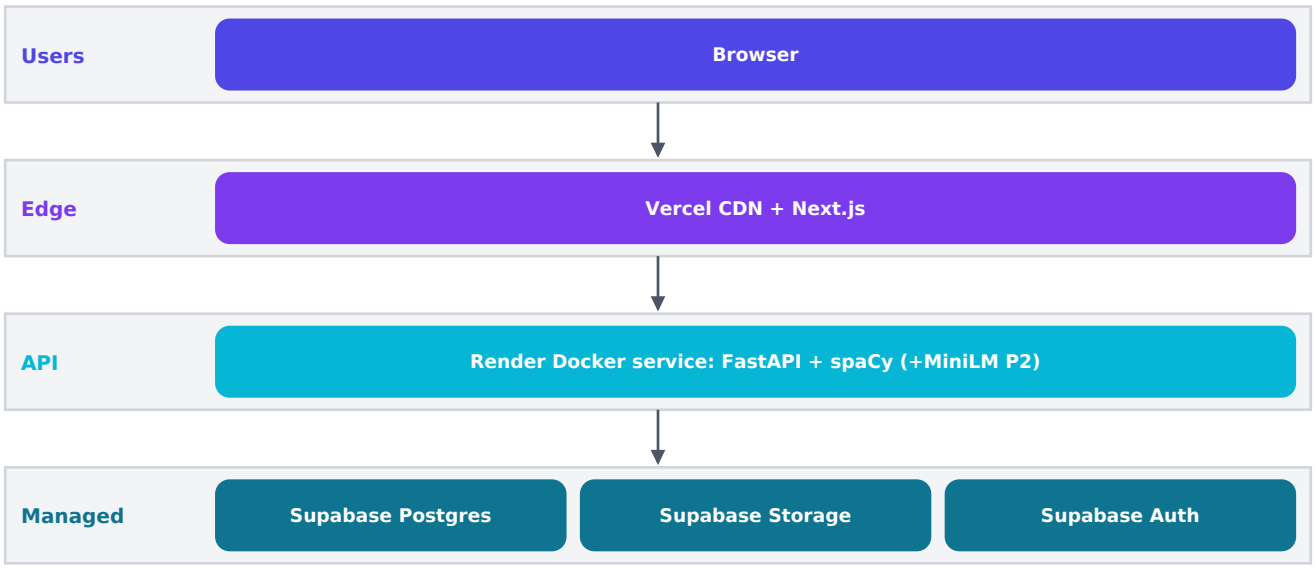


Figure 6. Deployment topology

Environments: `local`, `staging`, `production`. Details, environment variables, health checks and build/start commands are in the Deployment Specification.

14. Configuration Reference

Setting	Default	Description
MAX_TEXT_CHARS	150000	Maximum text length
MIN_TEXT_CHARS	200	Minimum text length
MAX_PDF_MB	10	Maximum upload size
MAX_PDF_PAGES	200	Maximum pages
MAX_TOPICS	25	Topics returned
WEIGHT_TFIDF / FREQUENCY / POSITION / SEMANTIC	0.40 / 0.25 / 0.20 / 0.15	Configured weights; renormalized over enabled signals
PRIORITY_HIGH_MIN / PRIORITY_MEDIUM_MIN	80 / 50	Priority thresholds
ENABLE_SEMANTIC, ENABLE_OCR, ENABLE_LLM	false, false, false	Feature flags (Phase 2 features)
CORS_ORIGINS	http://localhost:3000	Comma-separated allow-list
RATE_LIMIT_ANALYZE	10/minute	Anonymous analyze limit

14.1 Numeric Limits

Limit	Value	Error code
Minimum text length	200 characters (about 30 words)	TEXT_TOO_SHORT
Maximum text length (paste or extracted)	150,000 characters (about 25,000 words)	DOCUMENT_TOO_LARGE
Maximum PDF file size	10 MB	FILE_TOO_LARGE
Maximum PDF pages	200 pages	DOCUMENT_TOO_LARGE
Accepted upload types	application/pdf only (MVP); .txt and .md are Phase 2	INVALID_FILE_TYPE
Topics returned per analysis	Default 25, configurable 5 to 40	-
Analyze rate limit (anonymous)	10 requests per minute per IP	RATE_LIMITED
Analyze rate limit (authenticated, Phase 2)	30 requests per minute per user	RATE_LIMITED
Upload rate limit	5 requests per minute per IP (anonymous), 15 per user	RATE_LIMITED